Field-level encryption with a manifest

Selected fields are replaced in place. The manifest binds the set of encrypted values, so deleting or replaying one field is detected.

ENCRYPTED RESOURCE, AS STORED

```
{
"resourceType": "Condition",
"subject": { "reference": "Patient/8f3a" },
"code": {
"coding": [{ "code": "44054006" }],
"text":  "cpdpm2:AwCDUE0C…" ← code.text
},
"note": [
{ "text":    "cpdpm2:Q1BETQIB…" ← note[*].text },
{ "text":    "cpdpm2:Q1BETQIB…" ← note[*].text }
],
"meta": { "tag": [
{ "system": "urn:cpdpm:encrypted-paths", "code": "code.text" },
{ "system": "urn:cpdpm:encrypted-paths", "code": "note[*].text" },
{ "system": "urn:cpdpm:manifest",
        "code": "cpdpm2:Q1BETQIB…" } ← _manifest
] }
}
```

THE MANIFEST, ONCE DECRYPTED

```
{ "v": 1,
"paths": ["code.text", "note[*].text"],
"entries": [ ["code.text", "sha256:9f1c…"],
["note[*].text", "sha256:0be7…"],
["note[*].text", "sha256:c41a…"] ],
"clear": null }
```

Encrypted under Context(tenant, patient, "Condition", "_manifest").
With bind_cleartext=True, "clear" holds a digest of every unencrypted field.

WHAT decrypt_resource CHECKS, IN ORDER

1 Exactly one manifest tag. Decrypt it under the caller's context.

2 Walk each manifest path; every value must be a cpdpm2 string.

3 (path, digest) list must equal the manifest's entries exactly.

4 If "clear" is set, the clear-field digest must match too.

5 Only then decrypt each field under its own context.

Any failure → DecryptionError. The encrypted-paths tags are never trusted.

DETECTED delete a note · reorder notes · duplicate a note · put back an older ciphertext of one field
add a genuine ciphertext from another document · edit a clear field (when bound)

NOT DETECTED replacing the whole resource, manifest included, with an older version of itself. Needs a version in the application's context.